Лабораторная работа № 4

Эмуляция и измерение задержек в глобальных сетях

Авдадаев Джамал Геланиевич

Дисциплина: Моделирование сетей передачи данных

Mininet · Linux NETEM · Python · Gnuplot · Make

ОКТЯБРЬ 2026

Цель и экспериментальный подход

- Проверить влияние искусственной задержки на RTT виртуальной сети.
- Исследовать изменение задержки, джиттер, корреляцию и нормальное распределение.
- Автоматизировать запуск измерений, построение графиков и расчёт статистики.

> *Главное: сравниваются реальные измерения ping, а не только заданные параметры NETEM.*

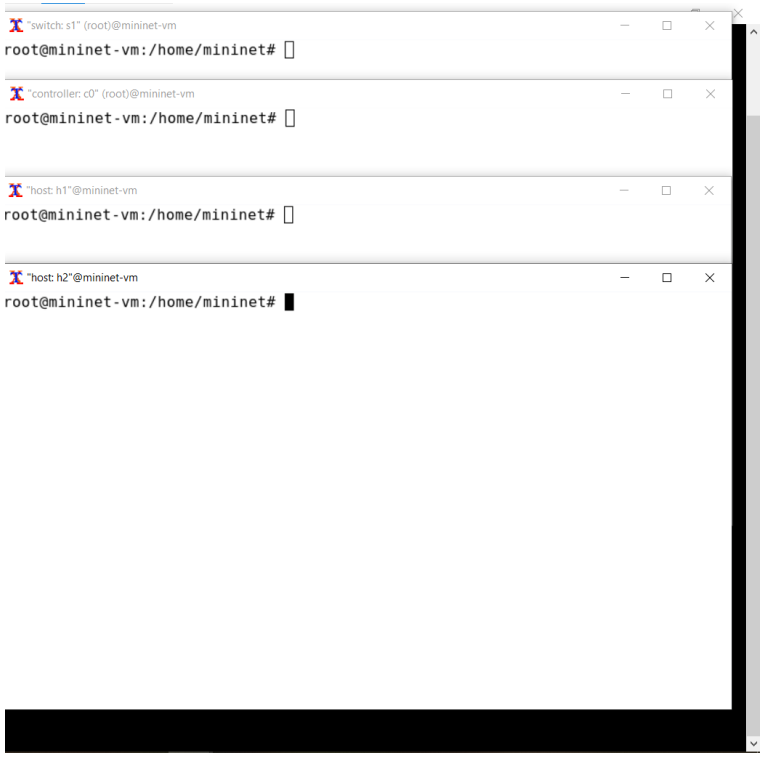
Топология виртуальной сети

h1
10.0.0.1

s1
коммутатор

h2
10.0.0.2

- Один коммутатор s1 и контроллер c0.
- NETEM действует на исходящий интерфейс; задержка на двух хостах складывается в RTT.



Запущенная сеть Mininet и терминалы хостов

Как управляли задержкой

Назначить задержку

sudo tc qdisc add dev h1-eth0 root netem delay 100ms

Изменить задержку

sudo tc qdisc change dev h1-eth0 root netem delay 50ms

Удалить правило

sudo tc qdisc del dev h1-eth0 root netem

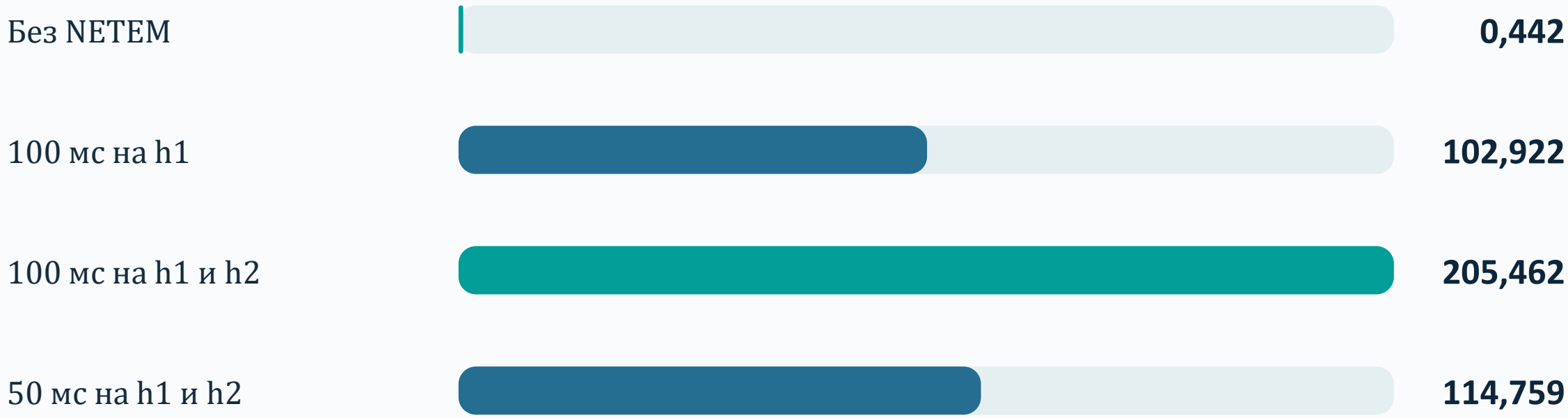
> *add — назначить задержку; change — изменить; del — удалить правило. Измерение после изменения параметров выполнялось командой ping.*

```
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=107 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=108 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010ms
rtt min/avg/max/mdev = 100.305/102.922/107.715/3.166 ms
root@mininet-vm:/home/mininet#
```

Проверка RTT после добавления задержки

Постоянная задержка меняет RTT



При задержке 100 мс на обоих исходящих интерфейсах RTT приблизился к 200 мс.

Интерактивные опыты со случайной задержкой

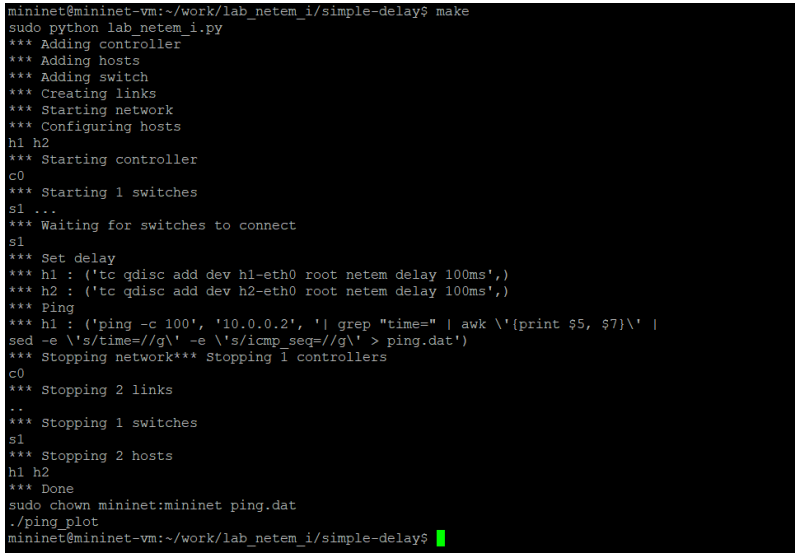
Режим	Число ответов	Средний RTT, мс	mdev, мс
100 мс, джиттер 10 мс	6	98.608	6.187
Корреляция 25 %	20	114.183	24.646
Нормальное распределение	10	109.870	20.757

> *Во всех показанных сериях потери ICMP-пакетов составили 0 %. Коэффициент корреляции задан в NETEM, но не вычислен из серии RTT.*

От ручного опыта к воспроизводимому

Mininet → NETEM → 100 × ping → Gnuplot → Статистика

- ping -c 100 формирует серию RTT в ping.dat.
- Gnuplot строит ping.png.
- ping_stats.py вычисляет min, avg, max и стандартное отклонение.



Автоматическое выполнение эксперимента командой make

simple-delay: постоянная задержка

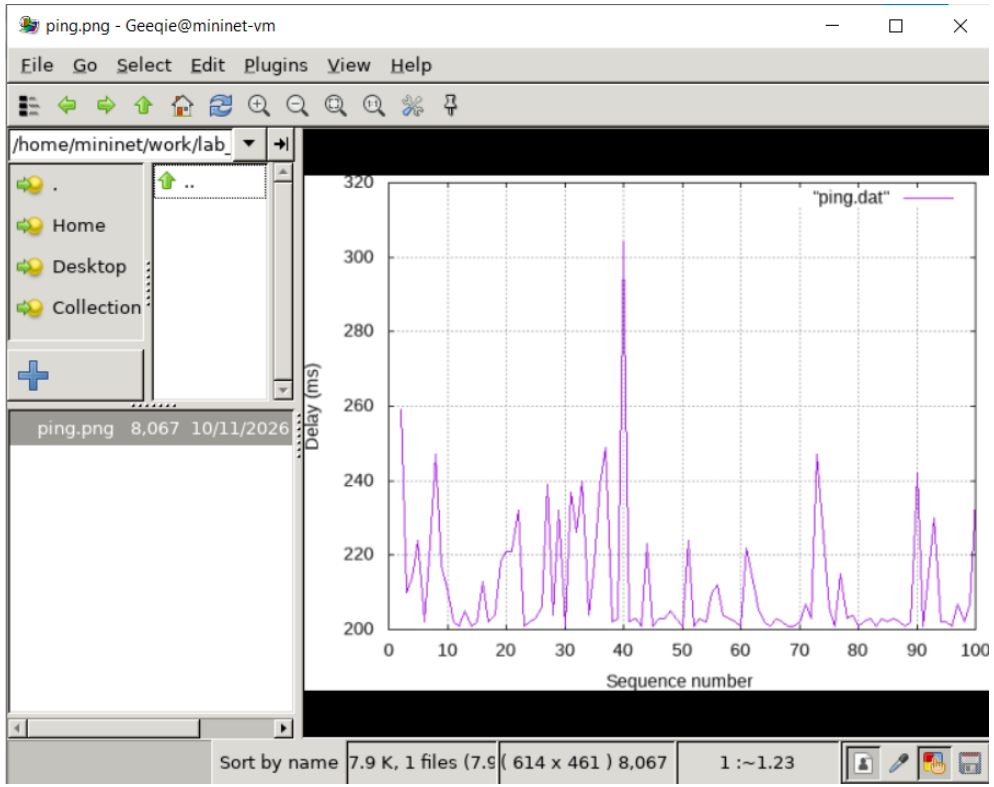


График RTT без первого измерения

Метрика	Значение
Средний RTT	211.838 мс
Мин. / макс.	201 / 304 мс
Ст. отклонение	16.522 мс

- 100 мс на h1 и 100 мс на h2.
- Первый выброс исключён по заданию методички.
- После удаления первого значения осталось 99 измерений.

change-delay: сравнение двух режимов

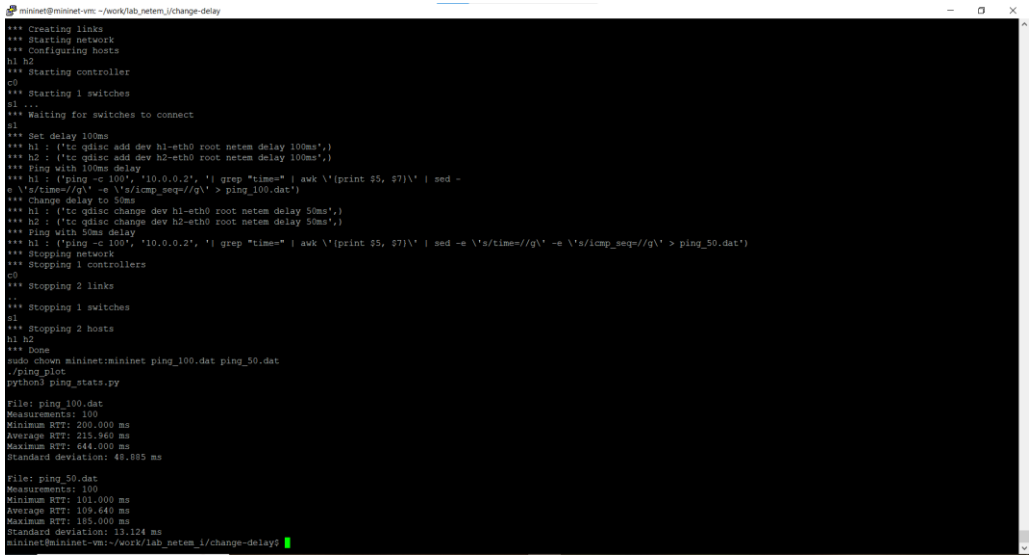
СРЕДНИЙ RTT — 100 мс / хост

215.960 мс

СРЕДНИЙ RTT — 50 мс / хост

109.640 мс

При уменьшении задержки на каждом хосте вдвое средний RTT сократился с 215,96 до 109,64 мс.



Запуск двух серий change-delay и расчёт статистики

jitter-delay: дрожание задержки 10 мс

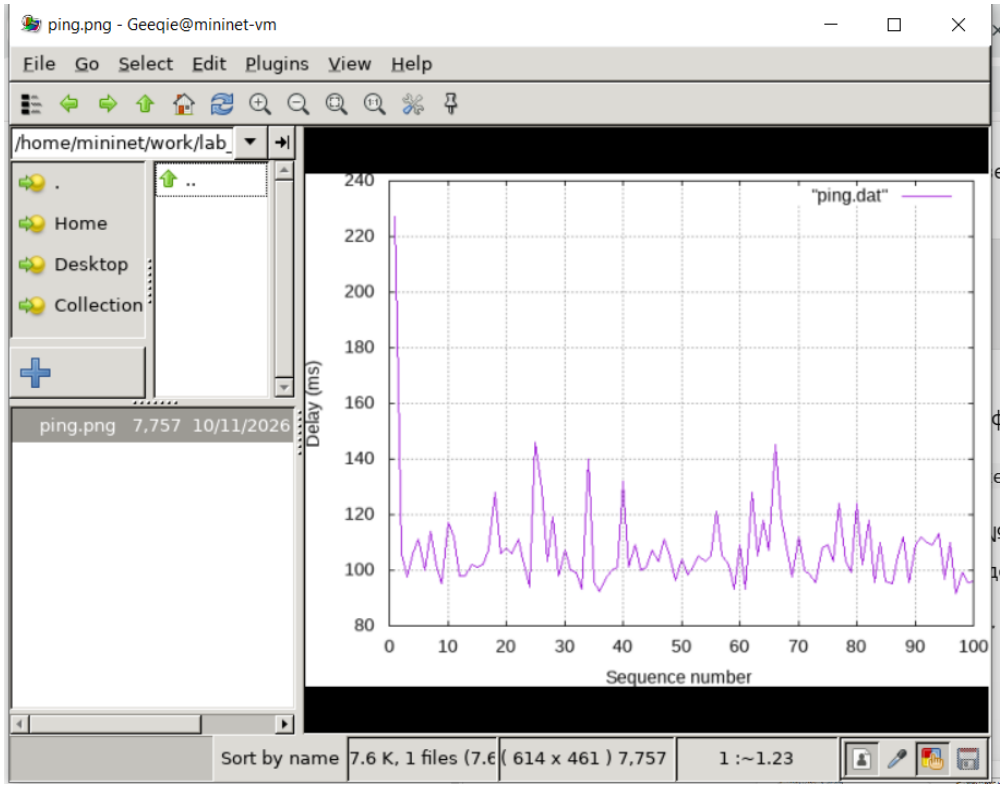
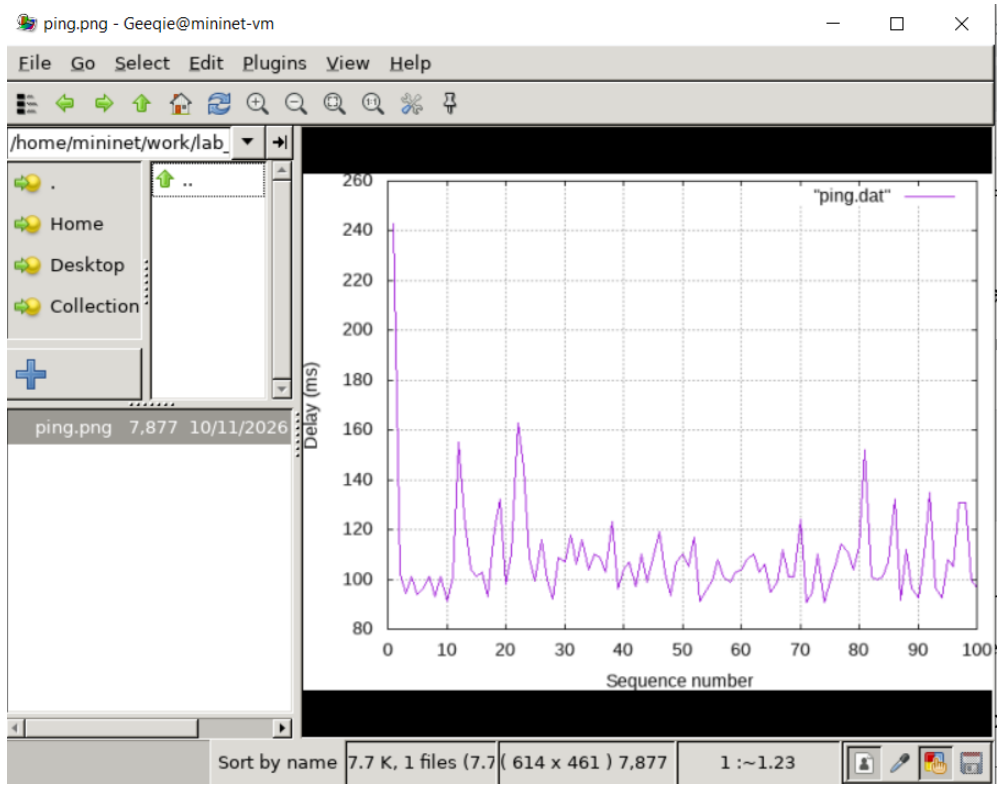


График джиттера NETEM

Метрика	Значение
Средний RTT	107.499 мс
Мин. / макс.	91.7 / 227 мс
Ст. отклонение	16.206 мс

- На h1 задано delay 100ms 10ms.
- Проведено 100 измерений RTT.
- Большинство значений сгруппировано около 100 мс; заметны отдельные пики.

correlation-delay: корреляция 25 %

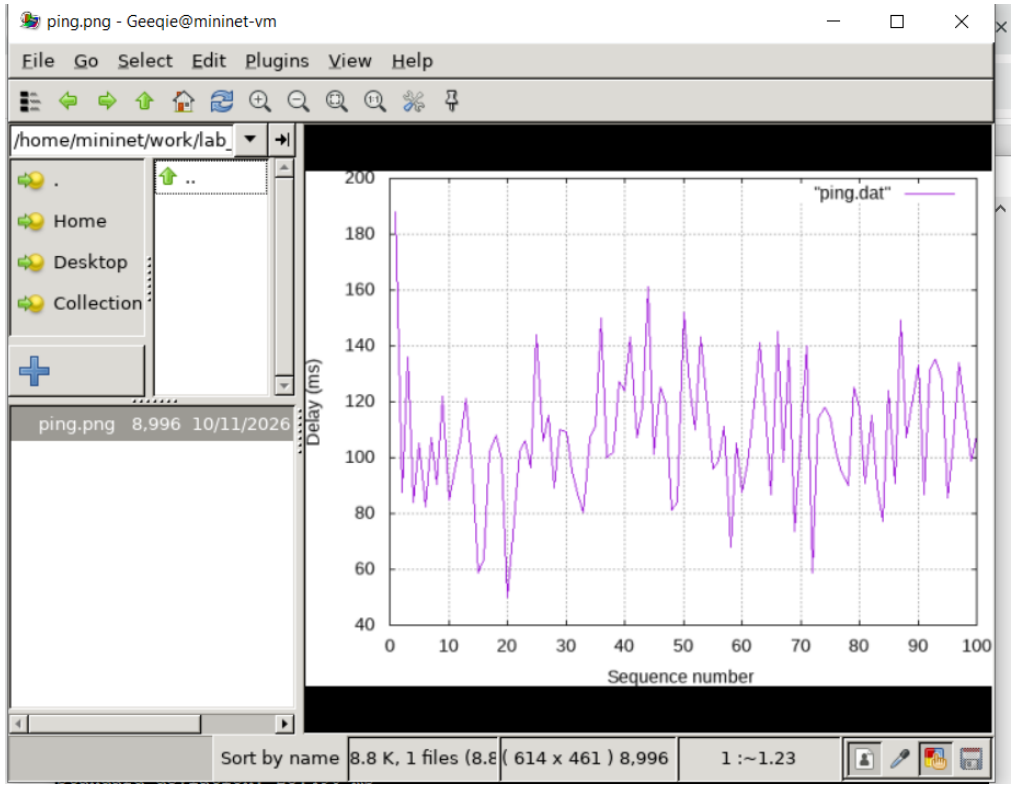


Ряд RTT при корреляции задержек 25 процентов

Метрика	Значение
Средний RTT	108.485 мс
Мин. / макс.	90.8 / 243 мс
Ст. отклонение	19.297 мс

- Заданы delay 100ms 10ms 25% на h1.
- Проведено 100 измерений RTT.
- По одному графику нельзя подтвердить точную фактическую корреляцию.

distribution-delay: нормальное распределение



Ряд RTT при нормальном распределении задержки

Метрика	Значение
Средний RTT	107.960 мс
Мин. / макс.	49.9 / 188 мс
Ст. отклонение	23.490 мс

- Использована настройка delay 100ms 20ms distribution normal.
- Проведено 100 измерений RTT.
- Наблюдаются отклонения RTT как выше, так и ниже базовых 100 мс.

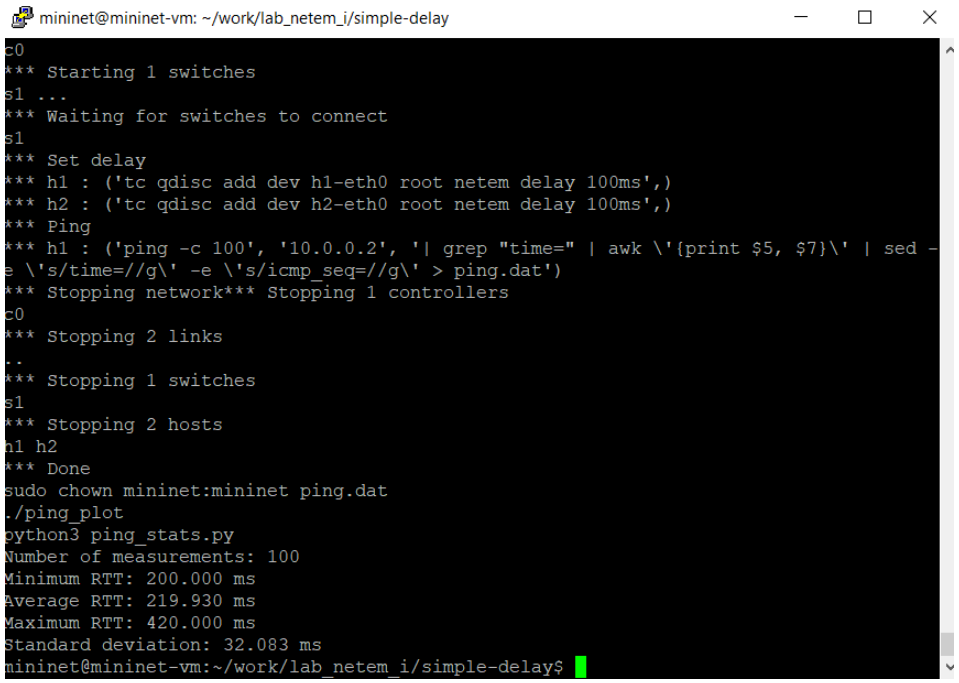
Сопоставление разброса RTT



В этих конкретных сериях большее стандартное отклонение получено при нормальном распределении. Это наблюдение, а не доказательство закона распределения.

Повторяемость эксперимента

- Пять независимых каталогов: simple-delay, change-delay, jitter-delay, correlation-delay, distribution-delay.
- В каждом: Python-скрипт, Makefile, данные RTT, график PNG и статистика.
- make запускает измерение и строит результаты; make clean очищает производные файлы.
- Исходный проект размещён в labs/lab04/project.



Повторный успешный запуск simple-delay командой make

ВЫВОДЫ

Основные выводы

01 NETEM позволяет воспроизводимо моделировать задержку и её изменчивость.

02 Два ограничения по 100 мс дали средний RTT 205,462 мс; без NETEM — 0,442 мс.

03 Уменьшение задержки до 50 мс на хост снизило средний RTT в автоматическом опыте примерно вдвое.

04 Python, GNU Make и Gnuplot обеспечили повторяемость измерений и сохранение результатов.

Главный результат: настроенная задержка проявляется в измеренном RTT, а автоматизация делает эксперимент проверяемым и повторяемым.